

Chapter 11

The Semistructured-Data Model

We now turn to a different kind of data model. This model, called “semistructured,” is distinguished by the fact that the schema is implied by the data, rather than being declared separately from the data as is the case for the relational model and all the other models we studied up to this point. After a general discussion of semistructured data, we turn to the most important manifestation of this idea: XML. We shall cover ways to describe XML data, in effect enforcing a schema for this “schemaless” data. These methods include DTD’s (Document Type Definitions) and the language XML Schema.

11.1 Semistructured Data

The *semistructured-data* model plays a special role in database systems:

1. It serves as a model suitable for *integration* of databases, that is, for describing the data contained in two or more databases that contain similar data with different schemas.
2. It serves as the underlying model for notations such as XML, to be taken up in Section 11.2, that are being used to share information on the Web.

In this section, we shall introduce the basic ideas behind “semistructured data” and how it can represent information more flexibly than the other models we have met previously.

11.1.1 Motivation for the Semistructured-Data Model

The models we have seen so far — E/R, UML, relational, ODL — each start with a schema. The schema is a rigid framework into which data is placed. This

rigidity provides certain advantages. Especially, the relational model owes much of its success to the existence of efficient implementations. This efficiency comes from the fact that the data in a relational database must fit the schema, and the schema is known to the query processor. For instance, fixing the schema allows the data to be organized with data structures that support efficient answering of queries, as we discussed in Section 8.3.

On the other hand, interest in the semistructured-data model is motivated primarily by its flexibility. In particular, semistructured data is “schemaless.” More precisely, the data is *self-describing*; it carries information about what its schema is, and that schema can vary arbitrarily, both over time and within a single database.

One might naturally wonder whether there is an advantage to creating a database without a schema, where one could enter data at will, and attach to the data whatever schema information you felt was appropriate for that data. There are actually some small-scale information systems, such as Lotus Notes, that take the self-describing-data approach. This flexibility may make query processing harder, but it offers significant advantages to users. For example, we can maintain a database of movies in the semistructured model and add new attributes like “would I like to see this movie?” as we wish. The attributes do not need to have a value for all movies, or even for more than one movie. Likewise, we can add relationships like “homage to,” without having to change the schema or even represent the relationship in more than one pair of movies.

11.1.2 Semistructured Data Representation

A database of *semistructured data* is a collection of *nodes*. Each node is either a *leaf* or *interior*. Leaf nodes have associated data; the type of this data can be any atomic type, such as numbers and strings. Interior nodes have one or more arcs out. Each arc has a *label*, which indicates how the node at the head of the arc relates to the node at the tail. One interior node, called the *root*, has no arcs entering and represents the entire database. Every node must be reachable from the root, although the graph structure is not necessarily a tree.

Example 11.1: Figure 11.1 is an example of a semistructured database about stars and movies. We see a node at the top labeled *Root*; this node is the entry point to the data and may be thought of as representing all the information in the database. The central objects or entities — stars and movies in this case — are represented by nodes that are children of the root.

We also see many leaf nodes. At the far left is a leaf labeled **Carrie Fisher**, and at the far right is a leaf labeled 1977, for instance. There are also many interior nodes. Three particular nodes we have labeled *cf*, *mh*, and *sw*, standing for “Carrie Fisher,” “Mark Hamill,” and “Star Wars,” respectively. These labels are not part of the model, and we placed them on these nodes only so we would have a way of referring to the nodes, which otherwise would be nameless, in the text. We may think of node *sw*, for instance, as representing the concept “Star

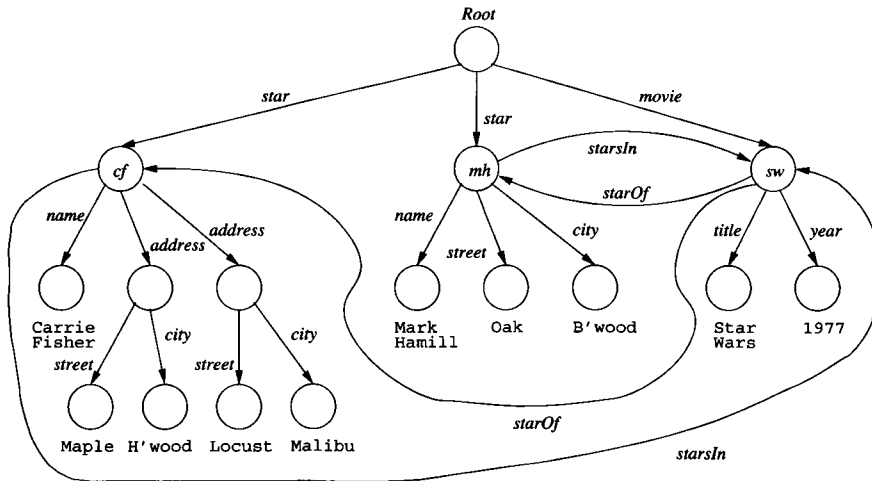


Figure 11.1: Semistructured data representing a movie and stars

Wars”: the title and year of this movie, other information not shown, such as its length, and its stars, two of which are shown. □

A label L on the arc from node N to node M can play one of two roles:

1. It may be possible to think of N as representing an object or entity, while M represents one of its attributes. Then, L represents the name of the attribute.
2. We may be able to think of N and M as objects or entities and L as the name of a relationship from N to M .

Example 11.2: Consider Fig. 11.1 again. The node indicated by cf may be thought of as representing the *Star* object for Carrie Fisher. We see, leaving this node, an arc labeled *name*, which represents the attribute *name* and leads to a leaf node holding the correct name. We also see two arcs, each labeled *address*. These arcs lead to unnamed nodes which we may think of as representing two addresses of Carrie Fisher. There is no schema to tell us whether stars can have more than one address; we simply put two address nodes in the graph if we feel it is appropriate.

Notice in Fig. 11.1 how both nodes have out-arcs labeled *street* and *city*. Moreover, these arcs each lead to leaf nodes with the appropriate atomic values. We may think of *address* nodes as structs or objects with two fields, named *street* and *city*. However, in the semistructured model, it is entirely appropriate to add other components, e.g., *zip*, to some addresses, or to have one or both fields missing.

The other kind of arc also appears in Fig. 11.1. For instance, the node *cf* has an out-arc leading to the node *sw* and labeled *starsIn*. The node *mh* (for Mark Hamill) has a similar arc, and the node *sw* has arcs labeled *starOf* to both nodes *cf* and *mh*. These arcs represent the stars-in relationship between stars and movies. □

11.1.3 Information Integration Via Semistructured Data

The flexibility and self-describing nature of semistructured data has made it important in two applications. We shall discuss its use for data exchange in Section 11.2, but here we shall consider its use as a tool for information integration. As databases have proliferated, it has become a common requirement that data in two or more of them be accessible as if they were one database. For instance, companies may merge; each has its own personnel database, its own database of sales, inventory, product designs, and perhaps many other matters. If corresponding databases had the same schemas, then combining them would be simple; for instance, we could take the union of the tuples in two relations that had the same schema and played the same roles in the the two databases.

However, life is rarely that simple. Independently developed databases are unlikely to share a schema, even if they talk about the same things, such as personnel. For instance, one employee database may record spouse-name, another not. One may have a way to represent several addresses, phones, or emails for an employee, another database may allow only one of each. One may treat consultants as employees, another not. One database might be relational, another object-oriented.

To make matters more complex, databases tend over time to be used in so many different applications that it is impossible to turn them off and copy or translate their data into another database, even if we could figure out an efficient way to transform the data from one schema to another. This situation is often referred to as the *legacy-database problem*; once a database has been in existence for a while, it becomes impossible to disentangle it from the applications that grow up around it, so the database can never be decommissioned.

A possible solution to the legacy-database problem is suggested in Fig. 11.2. We show two legacy databases with an interface; there could be many legacy systems involved. The legacy systems are each unchanged, so they can support their usual applications.

For flexibility in integration, the interface supports semistructured data, and the user is allowed to query the interface using a query language that is suitable for such data. The semistructured data may be constructed by translating the data at the sources, using components called *wrappers* (or “adapters”) that are each designed for the purpose of translating one source to semistructured data.

Alternatively, the semistructured data at the interface may not exist at all. Rather, the user queries the interface as if there were semistructured data, while the interface answers the query by posing queries to the sources, each referring to the schema found at that source.

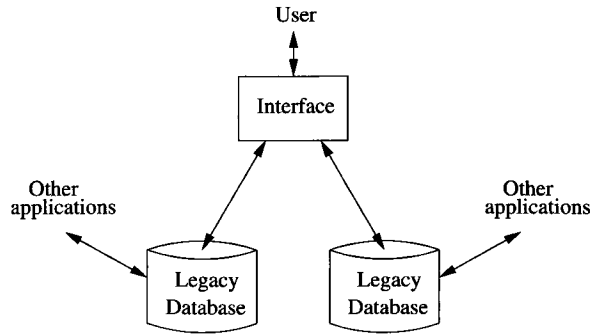


Figure 11.2: Integrating two legacy databases through an interface that supports semistructured data

Example 11.3: We can see in Fig. 11.1 a possible effect of information about stars being gathered from several sources. Notice that the address information for Carrie Fisher has an address concept, and the address is then broken into street and city. That situation corresponds roughly to data that had a nested-relation schema like `Stars(name, address(street, city))`.

On the other hand, the address information for Mark Hamill has no address concept at all, just street and city. This information may have come from a schema such as `Stars(name, street, city)` that can represent only one address for a star. Some of the other variations in schema that are not reflected in the tiny example of Fig. 11.1, but that could be present if movie information were obtained from several sources, include: optional film-type information, a director, a producer or producers, the owning studio, revenue, and information on where the movie is currently playing. □

11.1.4 Exercises for Section 11.1

Exercise 11.1.1: Since there is no schema to design in the semistructured-data model, we cannot ask you to design schemas to describe different situations. Rather, in the following exercises we shall ask you to suggest how particular data might be organized to reflect certain facts.

- a) Add to Fig. 11.1 the facts that *Star Wars* was directed by George Lucas and produced by Gary Kurtz.
- b) Add to Fig. 11.1 information about *Empire Strikes Back* and *Return of the Jedi*, including the facts that Carrie Fisher and Mark Hamill appeared in these movies.
- c) Add to (b) information about the studio (Fox) for these movies and the address of the studio (Hollywood).

Exercise 11.1.2: Suggest how typical data about banks and customers, as in Exercise 4.1.1, could be represented in the semistructured model.

Exercise 11.1.3: Suggest how typical data about players, teams, and fans, as was described in Exercise 4.1.3, could be represented in the semistructured model.

Exercise 11.1.4: Suggest how typical data about a genealogy, as was described in Exercise 4.1.6, could be represented in the semistructured model.

! Exercise 11.1.5: UML and the semistructured-data model are both “graphical” in nature, in the sense that they use nodes, labels, and connections among nodes as the medium of expression. Yet there is an essential difference between the two models. What is it?

11.2 XML

XML (*Extensible Markup Language*) is a tag-based notation designed originally for “marking” documents, much like the familiar HTML. Nowadays, data with XML “markup” can be represented in many ways. However, in this section we shall refer to XML data as represented in one or more documents. While HTML’s tags talk about the presentation of the information contained in documents — for instance, which portion is to be displayed in italics or what the entries of a list are — XML tags are intended to talk about the meanings of pieces of the document.

In this section we shall introduce the rudiments of XML. We shall see that it captures, in a linear form, the same structure as do the graphs of semistructured data introduced in Section 11.1. In particular, tags can play the same role as the labels on the arcs of a semistructured-data graph.

11.2.1 Semantic Tags

Tags in XML are text surrounded by triangular brackets, i.e., `<...>`, as in HTML. Also as in HTML, tags generally come in matching pairs, with an *opening tag* like `<Foo>` and a matched *closing tag* that is the same word with a slash, like `</Foo>`. Between a matching pair `<Foo>` and `</Foo>`, there can be text, including text with nested HTML tags, and any number of other nested matching pairs of XML tags. A pair of matching tags and everything that comes between them is called an *element*.

A single tag, with no matched closing tag, is also permitted in XML. In this form, the tag has a slash before the right bracket, for example, `<Foo/>`. Such a tag cannot have any other elements or text nested within it. It can, however, have attributes (see Section 11.2.4).

11.2.2 XML With and Without a Schema

XML is designed to be used in two somewhat different modes:

1. *Well-formed* XML allows you to invent your own tags, much like the arc-labels in semistructured data. This mode corresponds quite closely to semistructured data, in that there is no predefined schema, and each document is free to use whatever tags the author of the document wishes. Of course the nesting rule for tags must be obeyed, or the document is not well-formed.
2. *Valid* XML involves a “DTD,” or “Document Type Definition” (see Section 11.3) that specifies the allowable tags and gives a grammar for how they may be nested. This form of XML is intermediate between the strict-schema models such as the relational model, and the completely schemaless world of semistructured data. As we shall see in Section 11.3, DTD’s generally allow more flexibility in the data than does a conventional schema; DTD’s often allow optional fields or missing fields, for instance.

11.2.3 Well-Formed XML

The minimal requirement for well-formed XML is that the document begin with a declaration that it is XML, and that it have a *root element* that is the entire body of the text. Thus, a well-formed XML document would have an outer structure like:

```
<? xml version = "1.0" encoding = "utf-8" standalone = "yes" ?>
<SomeTag>
    . . .
</SomeTag>
```

The first line indicates that the file is an XML document. The encoding UTF-8 (UTF = “Unicode Transformation Format”) is a common choice of encoding for characters in documents, because it is compatible with ASCII and uses only one byte for the ASCII characters. The attribute `standalone = "yes"` indicates that there is no DTD for this document; i.e., it is well-formed XML. Notice that this initial declaration is delineated by special markers `<?...?>`. The root element for this document is labeled `<SomeTag>`.

Example 11.4: In Fig. 11.3 is an XML document that corresponds roughly to the data in Fig. 11.1. In particular, it corresponds to the tree-like portion of the semistructured data — the root and all the nodes and arcs except the “sideways” arcs among the nodes *cf*, *mh*, and *sw*. We shall see in Section 11.2.4 how those may be represented.

The root element is `StarMovieData`. Within this element, we see two elements, each beginning with the tag `<Star>` and ending with its matching

```
<? xml version = "1.0" encoding = "utf-8" standalone = "yes" ?>
<StarMovieData>
  <Star>
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
  </Star>
  <Star>
    <Name>Mark Hamill</Name>
    <Street>456 Oak Rd.</Street>
    <City>Brentwood</City>
  </Star>
  <Movie>
    <Title>Star Wars</Title>
    <Year>1977</Year>
  </Movie>
</StarMovieData>
```

Figure 11.3: An XML document about stars and movies

`</Star>`. Within each element is a subelement giving the name of the star. One element, for Carrie Fisher, also has two subelements, each giving the address of one of her homes. These elements are each delineated by an `<Address>` opening tag and its matched closing tag. The element for Mark Hamill has only subelements for one street and one city, and does not use an `<Address>` tag to group these. This distinction appeared as well in Fig. 11.1. We also see one element with opening tag `<Movie>` and its matched closing tag. This element has subelements for the title and year of the movie.

Notice that the document of Fig. 11.3 does not represent the relationship “stars-in” between stars and movies. We could indicate the movies of a star by including, within the element devoted to that star, the titles and years of their movies. Figure 11.4 is an example of this representation. □

11.2.4 Attributes

As in HTML, an XML element can have *attributes* (name-value pairs) within its opening tag. An attribute is an alternative way to represent a leaf node of semistructured data. Attributes, like tags, can represent labeled arcs in


```

<Star>
  <Name>Mark Hamill</Name>
  <Street>Oak</Street>
  <City>Brentwood</City>
  <Movie>
    <Title>Star Wars</Title>
    <Year>1977</Year>
  </Movie>
  <Movie>
    <Title>Empire Strikes Back</Title>
    <Year>1980</Year>
  </Movie>
</Star>

```

Figure 11.4: Nesting movies within stars

a semistructured-data graph. Attributes can also be used to represent the “sideways” arcs as in Fig. 11.1.

Example 11.5: The *title* or *year* children of the movie node labeled *sw* could be represented directly in the `<Movie>` element, rather than being represented by nested elements. That is, we could replace the `<Movie>` element of Fig. 11.3 by:

```
<Movie year = 1977><Title>Star Wars</Title></Movie>
```

We could even make both child nodes be attributes by:

```
<Movie title = "Star Wars" year = 1977></Movie>
```

or even:

```
<Movie title = "Star Wars" year = 1977 />
```

Notice that here we use a single tag without a matched closing tag, as indicated by the slash at the end. □

11.2.5 Attributes That Connect Elements

An important use for attributes is to represent connections in a semistructured data graph that do not form a tree. We shall see in Section 11.3.4 how to declare certain attributes to be identifiers for their elements. We shall also see how to declare that other attributes are references to these element identifiers. For the moment, let us just see an example of how these attributes could be used.

Example 11.6: Figure 11.5 can be interpreted as an exact representation in XML of the semistructured data graph of Fig. 11.1. However, in order to make the interpretation, we need to have enough schema information that we know the attribute `starID` is an identifier for the element in which it appears. That is, `cf` is the identifier of the first `<Star>` element (for Carrie Fisher) and `mh` is the identifier of the second `<Star>` element (for Mark Hamill). Likewise, we must establish that the attribute `movieID` within a `<Movie>` tag is an identifier for that element. Thus, `sw` is an identifier for the lone `<Movie>` element in Fig. 11.5.

```
<? xml version = "1.0" encoding = "utf-8" standalone = "yes" ?>
<StarMovieData>
  <Star starID = "cf" starredIn = "sw">
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
  </Star>
  <Star starID = "mh" starredIn = "sw">
    <Name>Mark Hamill</Name>
    <Street>456 Oak Rd.</Street>
    <City>Brentwood</City>
  </Star>
  <Movie movieID = "sw" starsOf = "cf", "mh">
    <Title>Star Wars</Title>
    <Year>1977</Year>
  </Movie>
</StarMovieData>
```

Figure 11.5: Adding stars-in information to our XML document

Moreover, the schema must also say that the attributes `starredIn` for `<Star>` elements and `starsOf` for `<Movie>` elements are references to one or more ID's. That is, the value `sw` for `starredIn` within each of the `<Movie>` elements says that both Carrie Fisher and Mark Hamill starred in *Star Wars*. Likewise, the list of ID's `cf` and `mh` that is the value of `starsOf` in the `<Movie>` element says that both these stars were stars of *Star Wars*. □

11.2.6 Namespaces

There are situations in which XML data involves tags that come from two or more different sources, and which may therefore have conflicting names. For example, we would not want to confuse an HTML tag used in text with an XML tag that represents the meaning of that text. In Section 11.4, we shall see how XML Schema requires tags from two separate vocabularies. To distinguish among different vocabularies for tags in the same document, we can use a *namespace* for a set of tags.

To say that an element's tag should be interpreted as part of a certain namespace, we can use the attribute `xmlns` in its opening tag. There is a special form used for this attribute:

```
xmlns:name="URI"
```

Within the element having this attribute, *name* can modify any tag to say the tag belongs to this namespace. That is, we can create *qualified* names of the form *name:tag*, where *name* is the name of the namespace to which the tag *tag* belongs.

The URI (Universal Resource Identifier) is typically a URL referring to a document that describes the meaning of the tags in the namespace. This description need not be formal; it could be an informal article about expectations. It could even be nothing at all, and still serve the purpose of distinguishing different tags that had the same name.

Example 11.7: Suppose we want to say that in element `StarMovieData` of Fig. 11.5 certain tags belong to the namespace defined in the document `infolab.stanford.edu/movies`. We could choose a name such as `md` for the namespace by using the opening tag:

```
<md:StarMovieData xmlns:md=
    "http://infolab.stanford.edu/movies">
```

Our intent is that `StarMovieData` itself is part of this namespace, so it gets the prefix `md:`, as does its closing tag `/md:StarMovieData`. Inside this element, we have the option of asserting that the tags of subelements belong to this namespace by prefixing their opening and closing tags with `md:`. □

11.2.7 XML and Databases

Information encoded in XML is not always intended to be stored in a database. It has become common for computers to share data across the Internet by passing messages in the form of XML elements. These messages live for a very short time, although they may have been generated using data from one database and wind up being stored as tuples of a database at the receiving end. For example, the XML data in Fig. 11.5 might be turned into some tuples

to insert into relations `MovieStar` and `StarsIn` of our running example movie database.

However, it is becoming increasingly common for XML to appear in roles traditionally reserved for relational databases. For example, we discussed in Section 11.1.3 how systems that integrate the data of an enterprise produce integrated views of many databases. XML is becoming an important option as the way to represent these views, as an alternative to views consisting of relations or classes of objects. The integrated views are then queried using one of the specialized XML query languages that we shall meet in Chapter 12.

When we store XML in a database, we must deal with the requirement that access to information must be efficient, especially for very large XML documents or very large collections of small documents.¹ A relational DBMS provides indexes and other tools for making access efficient, a subject we introduced in Section 8.3. There are two approaches to storing XML to provide some efficiency:

1. Store the XML data in a parsed form, and provide a library of tools to navigate the data in that form. Two common standards are called SAX (Simple API for XML) and DOM (Document Object Model).
2. Represent the documents and their elements as relations, and use a conventional, relational DBMS to store them.

In order to represent XML documents as relations, we should start by giving each document and each element of those documents a unique ID. For the document, the ID could be its URL or path in a file system. A possible relational database schema is:

```
DocRoot(docID, rootElementID)
SubElement(parentID, childID, position)
ElementAttribute(elementID, name, value)
ElementValue(elementID, value)
```

This schema is suitable for documents that obey the restriction that each element either contains only text or contains only subelements. Accommodating elements with *mixed content* of text and subelements is left as an exercise.

The first relation, `DocRoot` relates document ID's to the ID's of their root element. The second relation, `SubElement`, connects an element (the “parent”) to each of its immediate subelements (“children”). The third attribute of `SubElement` gives the position of the child among all the children of the parent.

The third relation, `ElementAttribute` relates elements to their attributes; each tuple gives the name and value of one of the attributes of an element. Finally, `ElementValue` relates those elements that have no subelements to the text, if any, that is contained in that element.

¹Recall that XML data need not take the form of documents (i.e., a header with a root element) at all. For example, XML data could be a stream of elements without headers. However, we shall continue to speak of “documents” as XML data.

There is a small matter that values of attributes and elements can have different types, e.g., integers or strings, while relational attributes each have a unique type. We could treat the two attributes named *value* as always being strings, and interpret those strings that were integers or another type properly as we processed the data. Or we could split each of the last two relations into as many relations as there are different types of data.

11.2.8 Exercises for Section 11.2

Exercise 11.2.1: Repeat Exercise 11.1.1 using XML.

Exercise 11.2.2: Show that any relation can be represented by an XML document. *Hint:* Create an element for each tuple with a subelement for each component of that tuple.

! **Exercise 11.2.3:** How would you represent an empty element (one that had neither text nor subelements) in the database schema of Section 11.2.7?

! **Exercise 11.2.4:** In Section 11.2.7 we gave a database schema for representing documents that do not have *mixed content* — elements that contain a mixture of text (#PCDATA) and subelements. Show how to modify the schema when elements can have mixed content.

11.3 Document Type Definitions

For a computer to process XML documents automatically, it is helpful for there to be something like a schema for the documents. It is useful to know what kinds of elements can appear in a collection of documents and how elements can be nested. The description of the schema is given by a grammar-like set of rules, called a *document type definition*, or DTD. It is intended that companies or communities wishing to share data will each create a DTD that describes the form(s) of the data they share, thus establishing a shared view of the semantics of their elements. For instance, there could be a DTD for describing protein structures, a DTD for describing the purchase and sale of auto parts, and so on.

11.3.1 The Form of a DTD

The gross structure of a DTD is:

```
<!DOCTYPE root-tag [
    <!ELEMENT element-name (components)>
    more elements
]>
```

The opening *root-tag* and its matched closing tag surround a document that conforms to the rules of this DTD. Element declarations, introduced by `!ELEMENT`, give the tag used to surround the portion of the document that represents the element, and also give a parenthesized list of “components.” The latter are elements that may or must appear in the element being described. The exact requirements on components are indicated in a manner we shall see shortly.

There are two important special cases of components:

1. `(#PCDATA)` (“parsed character data”) after an element name means that element has a value that is text, and it has no elements nested within. Parsed character data may be thought of as HTML text. It can have formatting information within it, and the special characters like `<` must be escaped, by `<`; and similar HTML codes. For instance,

```
<!ELEMENT Title (#PCDATA)>
```

says that between `<Title>` and `</Title>` tags a character string can appear. However, any nested tags are not part of the XML; they could be HTML, for instance.

2. The keyword `EMPTY`, with no parentheses, indicates that the element is one of those that has no matched closing tag. It has no subelements, nor does it have text as a value. For instance,

```
<!ELEMENT Foo EMPTY>
```

says that the only way the tag `Foo` can appear is as `<Foo/>`.

Example 11.8: In Fig. 11.6 we see a DTD for stars.² The DTD name and root element is `Stars`. The first element definition says that inside the matching pair of tags `<Stars>...</Stars>` we shall find zero or more `Star` elements, each representing a single star. It is the `*` in `(Star*)` that says “zero or more,” i.e., “any number of.”

The second element, `Star`, is declared to consist of three kinds of subelements: `Name`, `Address`, and `Movies`. They must appear in this order, and each must be present. However, the `+` following `Address` says “one or more”; that is, there can be any number of addresses listed for a star, but there must be at least one. The `Name` element is then defined to be parsed character data. The fourth element says that an address element consists of subelements for a street and a city, in that order.

Then, the `Movies` element is defined to have zero or more elements of type `Movie` within it; again, the `*` says “any number of.” A `Movie` element is defined to consist of title and year elements, each of which are simple text. Figure 11.7 is an example of a document that conforms to the DTD of Fig. 11.6. □

```

<!DOCTYPE Stars [
    <!ELEMENT Stars (Star*)>
    <!ELEMENT Star (Name, Address+, Movies)>
    <!ELEMENT Name (#PCDATA)>
    <!ELEMENT Address (Street, City)>
    <!ELEMENT Street (#PCDATA)>
    <!ELEMENT City (#PCDATA)>
    <!ELEMENT Movies (Movie*)>
    <!ELEMENT Movie (Title, Year)>
    <!ELEMENT Title (#PCDATA)>
    <!ELEMENT Year (#PCDATA)>
]>

```

Figure 11.6: A DTD for movie stars

The components of an element *E* are generally other elements. They must appear between the tags `<E>` and `</E>` in the order listed. However, there are several operators that control the number of times elements appear.

1. A `*` following an element means that the element may occur any number of times, including zero times.
2. A `+` following an element means that the element may occur one or more times.
3. A `?` following an element means that the element may occur either zero times or one time, but no more.
4. We can connect a list of options by the “or” symbol `|` to indicate that exactly one option appears. For example, if `<Movie>` elements had `<Genre>` subelements, we might declare these by

```
<!ELEMENT Genre (Comedy|Drama|SciFi|Teen)>
```

to indicate that each `<Genre>` element has one of these four subelements.

5. Parentheses can be used to group components. For example, if we declared addresses to have the form

```
<!ELEMENT Address Street, (City|Zip)>
```

then `<Address>` elements would each have a `<Street>` subelement followed by either a `<City>` or `<Zip>` subelement, but not both.

²Note that the stars-and-movies XML document of Fig. 11.3 is not intended to conform to this DTD.

```

<Stars>
  <Star>
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
    <Movies>
      <Movie>
        <Title>Star Wars</Title>
        <Year>1977</Year>
      </Movie>
      <Movie>
        <Title>Empire Strikes Back</Title>
        <Year>1980</Year>
      </Movie>
      <Movie>
        <Title>Return of the Jedi</Title>
        <Year>1983</Year>
      </Movie>
    </Movies>
  </Star>
  <Star>
    <Name>Mark Hamill</Name>
    <Address>
      <Street>456 Oak Rd.<Street>
      <City>Brentwood</City>
    </Address>
    <Movies>
      <Movie>
        <Title>Star Wars</Title>
        <Year>1977</Year>
      </Movie>
      <Movie>
        <Title>Empire Strikes Back</Title>
        <Year>1980</Year>
      </Movie>
      <Movie>
        <Title>Return of the Jedi</Title>
        <Year>1983</Year>
      </Movie>
    </Movies>
  </Star>
</Stars>

```

Figure 11.7: Example of a document following the DTD of Fig. 11.6

11.3.2 Using a DTD

If a document is intended to conform to a certain DTD, we can either:

- a) Include the DTD itself as a preamble to the document, or
- b) In the opening line, refer to the DTD, which must be stored separately in the file system accessible to the application that is processing the document.

Example 11.9: Here is how we might introduce the document of Fig. 11.7 to assert that it is intended to conform to the DTD of Fig. 11.6.

```
<?xml version = "1.0" encoding = "utf-8" standalone = "no"?>
<!DOCTYPE Stars SYSTEM "star.dtd">
```

The attribute `standalone = "no"` says that a DTD is being used. Recall we set this attribute to `"yes"` when we did not wish to specify a DTD for the document. The location from which the DTD can be obtained is given in the `!DOCTYPE` clause, where the keyword `SYSTEM` followed by a file name gives this location. □

11.3.3 Attribute Lists

A DTD also lets us specify which attributes an element may have, and what the types of these attributes are. A declaration of the form

```
<!ATTLIST element-name attribute-name type >
```

says that the named attribute can be an attribute of the named element, and that the type of this attribute is the indicated type. Several attributes can be defined in one `ATTLIST` statement, but it is not necessary to do so, and the `ATTLIST` statements can appear in any position in the DTD.

The most common type for attributes is `CDATA`. This type is essentially character-string data with special characters like `<` escaped as in `#PCDATA`. Notice that `CDATA` does not take a pound sign as `#PCDATA` does. Another option is an enumerated type, which is a list of possible strings, surrounded by parentheses and separated by `|`'s. Following the data type there can be a keyword `#REQUIRED` or `#IMPLIED`, which means that the attribute must be present, or is optional, respectively.

Example 11.10: Instead of having the title and year be subelements of a `<Movie>` element, we could make these be attributes instead. Figure 11.8 shows possible attribute-list declarations. Notice that `Movie` is now an empty element. We have given it three attributes: `title`, `year`, and `genre`. The first two are `CDATA`, while the `genre` has values from an enumerated type. Note that in the document, the values, such as *comedy*, appear with quotes. Thus,

```
<Movie title = "Star Wars" year = "1977" genre = "sciFi" />
```

is a possible movie element in a document that conforms to this DTD. □

```

<!ELEMENT Movie EMPTY>
  <!ATTLIST Movie
    title CDATA #REQUIRED
    year CDATA #REQUIRED
    genre (comedy | drama | sciFi | teen) #IMPLIED
  >

```

Figure 11.8: Data about movies will appear as attributes

```

<!DOCTYPE StarMovieData [
  <!ELEMENT StarMovieData (Star*, Movie*)>
  <!ELEMENT Star (Name, Address+)>
    <!ATTLIST Star
      starId ID #REQUIRED
      starredIn IDREFS #IMPLIED
    >
  <!ELEMENT Name (#PCDATA)>
  <!ELEMENT Address (Street, City)>
  <!ELEMENT Street (#PCDATA)>
  <!ELEMENT City (#PCDATA)>
  <!ELEMENT Movie (Title, Year)>
    <!ATTLIST Movie
      movieId ID #REQUIRED
      starsOf IDREFS #IMPLIED
    >
  <!ELEMENT Title (#PCDATA)>
  <!ELEMENT Year (#PCDATA)>
]>

```

Figure 11.9: A DTD for stars and movies, using ID's and IDREF's

11.3.4 Identifiers and References

Recall from Section 11.2.5 that certain attributes can be used as identifiers for elements. In a DTD, we give these attributes the type ID. Other attributes have values that are references to these element ID's; these attributes may be declared to have type IDREF. The value of an IDREF attribute must also be the value of some ID attribute of some element, so the IDREF is in effect a pointer to the ID. An alternative is to give an attribute the type IDREFS. In that case, the value of the attribute is a string consisting of a list of ID's, separated by whitespace. The effect is that an IDREFS attribute links its element to a set of elements — the elements identified by the ID's on the list.

Example 11.11: Figure 11.9 shows a DTD in which stars and movies are given equal status, and the ID-IDREFS correspondence is used to describe the many-many relationship between movies and stars that was suggested in the semistructured data of Fig. 11.1. The structure differs from that of the DTD in Fig. 11.6, in that stars and movies have equal status; both are subelements of the root element. That is, the name of the root element for this DTD is *StarMovieData*, and its elements are a sequence of stars followed by a sequence of movies.

A star no longer has a set of movies as subelements, as was the case for the DTD of Fig. 11.6. Rather, its only subelements are a name and address, and in the beginning `<Star>` tag we shall find an attribute *starredIn* of type IDREFS, whose value is a list of ID's for the movies of the star.

```
<? xml version = "1.0" encoding = "utf-8" standalone = "yes" ?>
<StarMovieData>
  <Star starID = "cf" starredIn = "sw">
    <Name>Carrie Fisher</Name>
    <Address>
      <Street>123 Maple St.</Street>
      <City>Hollywood</City>
    </Address>
    <Address>
      <Street>5 Locust Ln.</Street>
      <City>Malibu</City>
    </Address>
  </Star>
  <Star starID = "mh" starredIn = "sw">
    <Name>Mark Hamill</Name>
    <Address>
      <Street>456 Oak Rd.</Street>
      <City>Brentwood</City>
    </Address>
  </Star>
  <Movie movieID = "sw" starsOf = "cf mh">
    <Title>Star Wars</Title>
    <Year>1977</Year>
  </Movie>
</StarMovieData>
```

Figure 11.10: Adding stars-in information to our XML document

A `<Star>` element also has an attribute *starId*. Since it is declared to be of type ID, the value of *starId* may be referenced by `<Movie>` elements to indicate the stars of the movie. That is, when we look at the attribute list for *Movie* in Fig. 11.9, we see that it has an attribute *movieId* of type ID; these

are the ID's that will appear on lists that are the values of `starredIn` elements. Symmetrically, the attribute `starsOf` of `Movie` is an IDREFS, a list of ID's for stars. □

11.3.5 Exercises for Section 11.3

Exercise 11.3.1: Add to the document of Fig. 11.10 the following facts:

- a) Carrie Fisher and Mark Hamill also starred in *The Empire Strikes Back* (1980) and *Return of the Jedi* (1983).
- b) Harrison Ford also starred in *Star Wars*, in the two movies mentioned in (a), and the movie *Firewall* (2006).
- c) Carrie Fisher also starred in *Hannah and Her Sisters* (1985).
- d) Matt Damon starred in *The Bourne Identity* (2002).

Exercise 11.3.2: Suggest how typical data about banks and customers, as was described in Exercise 4.1.1, could be represented as a DTD.

Exercise 11.3.3: Suggest how typical data about players, teams, and fans, as was described in Exercise 4.1.3, could be represented as a DTD.

Exercise 11.3.4: Suggest how typical data about a genealogy, as was described in Exercise 4.1.6, could be represented as a DTD.

! Exercise 11.3.5: Using your representation from Exercise 11.2.2, devise an algorithm that will take any relation schema (a relation name and a list of attribute names) and produce a DTD describing a document that represents that relation.

11.4 XML Schema

XML Schema is an alternative way to provide a schema for XML documents. It is more powerful than DTD's, giving the schema designer extra capabilities. For instance, XML Schema allows arbitrary restrictions on the number of occurrences of subelements. It allows us to declare types, such as integer or float, for simple elements, and it gives us the ability to declare keys and foreign keys.

11.4.1 The Form of an XML Schema

An XML Schema description of a schema is itself an XML document. It uses the namespace at the URL:

<http://www.w3.org/2001/XMLSchema>

that is provided by the World-Wide-Web Consortium. Each XML-Schema document thus has the form:

```
<? xml version = "1.0" encoding = "utf-8" ?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
    ...
</xs:schema>
```

The first line indicates XML, and uses the special brackets `<? amd ?>`. The second line is the root tag for the document that is the schema. The attribute `xmlns` (XML namespace) makes the variable `xs` stand for the namespace for XML Schema that was mentioned above. It is this namespace that causes the tag `<xs:schema>` to be interpreted as `schema` in the namespace for XML Schema. As discussed in Section 11.2.6, qualifying each XML-Schema term we use with the prefix `xs:` will cause each such tag to be interpreted according to the rules for XML Schema. Between the opening `<xs:schema>` tag and its matched closing tag `</xs:schema>` will appear a schema. In what follows, we shall learn the most important tags from the XML-Schema namespace and what they mean.

11.4.2 Elements

An important component of schemas is the *element*, which is similar to an element definition in a DTD. In the discussion that follows, you should be alert to the fact that, because XML-Schema definitions are XML documents, these schemas are themselves composed of “elements.” However, the elements of the schema itself, each of which has a tag that begins with `xs:`, are not the elements being defined by the schema.³ The form of an element definition in XML Schema is:

```
<xs:element name = element name type = element type >
    constraints and/or structure information
</xs:element>
```

The element name is the chosen tag for these elements in the schema being defined. The type can be either a simple type or a complex type. Simple types include the common primitive types, such as `xs:integer`, `xs:string`, and `xs:boolean`. There can be no subelements for an element of a simple type.

Example 11.12: Here are title and year elements defined in XML Schema:

```
<xs:element name = "Title" type = "xs:string" />
<xs:element name = "Year" type = "xs:integer" />
```

³To further assist in the distinction between tags that are part of a schema definition and the tags of the schema being defined, we shall begin each of the latter with a capital letter.

Each of these `<xs:element>` elements is itself empty, so it can be closed by `/>` with no matched closing tag. The first defined element has name `Title` and is of string type. The second element is named `Year` and is of type integer. In documents (perhaps talking about movies) with `<Title>` and `<Year>` elements, these elements will not be empty, but rather will be followed by a string (the title) or integer (the year), and a matched closing tag, `</Title>` or `</Year>`, respectively. $\square$

11.4.3 Complex Types

A *complex type* in XML Schema can have several forms, but the most common is a sequence of elements. These elements are required to occur in the sequence given, but the number of repetitions of each element can be controlled by attributes `minOccurs` and `maxOccurs`, that appear in the element definitions themselves. The meanings of these attributes are as expected; no fewer than `minOccurs` occurrences of each element may appear in the sequence, and no more than `maxOccurs` occurrences may appear. If there is more than one occurrence, they must all appear consecutively. The default, if one or both of these attributes are missing, is one occurrence. To say that there is no upper limit on occurrences, use the value "unbounded" for `maxOccurs`.

```
<xs:complexType name = type name >
  <xs:sequence>
    list of element definitions
  </xs:sequence>
</xs:complexType>
```

Figure 11.11: Defining a complex type that is a sequence of elements

The form of a definition for a complex-type that is a sequence of elements is shown in Fig. 11.11. The name for the complex type is optional, but is needed if we are going to use this complex type as the type of one or more elements of the schema being defined. An alternative is to place the complex-type definition between an opening `<xs:element>` tag and its matched closing tag, to make that complex type be the type of the element.

Example 11.13: Let us write a complete XML-Schema document that defines a very simple schema for movies. The root element for movie documents will be `<Movies>`, and the root will have zero or more `<Movie>` subelements. Each `<Movie>` element will have two subelements: a title and year, in that order. The XML-Schema document is shown in Fig. 11.12.

Lines (1) and (2) are a typical preamble to an XML-Schema definition. In lines (3) through (8), we define a complex type, whose name is `movieType`. This type consists of a sequence of two elements named `Title` and `Year`; they are the elements we saw in Example 11.12. The type definition itself does not

```

1) <? xml version = "1.0" encoding = "utf-8" ?>
2) <xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">

3)     <xs:complexType name = "movieType">
4)         <xs:sequence>
5)             <xs:element name = "Title" type = "xs:string" />
6)             <xs:element name = "Year" type = "xs:integer" />
7)         </xs:sequence>
8)     </xs:complexType>

9)     <xs:element name = "Movies">
10)         <xs:complexType>
11)             <xs:sequence>
12)                 <xs:element name = "Movie" type = "movieType"
13)                     minOccurs = "0" maxOccurs = "unbounded" />
14)             </xs:sequence>
15)         </xs:complexType>
16)     </xs:element>

16) </xs:schema>

```

Figure 11.12: A schema for movies in XML Schema

create any elements, but notice how the name `movieType` is used in line (12) to make this type be the type of `Movie` elements.

Lines (9) through (15) define the element `Movies`. Although we could have created a complex type for this element, as we did for `Movie`, we have chosen to include the type in the element definition itself. Thus, we put no type attribute in line (9). Rather, between the opening `<xs:element>` tag at line (9) and its matched closing tag at line (15) appears a complex-type definition for the element `Movies`. This complex type has no name, but it is defined at line (11) to be a sequence. In this case, the sequence has only one kind of element, `Movie`, as indicated by line (12). This element is defined to have type `movieType` — the complex type we defined at lines (3) through (8). It is also defined to have between zero and infinity occurrences. Thus, the schema of Fig. 11.12 says the same thing as the DTD we show in Fig. 11.13. □

There are several other ways we can construct a complex type.

- In place of `xs:sequence` we could use `xs:all`, which means that each of the elements between the opening `<xs:all>` tag and its matched closing tag must occur, in any order, exactly once each.
- Alternatively, we could replace `xs:sequence` by `xs:choice`. Then, exactly one of the elements found between the opening `<xs:choice>` tag

```

<!DOCTYPE Movies [
    <!ELEMENT Movies (Movie*)>
    <!ELEMENT Movie (Title, Year)>
    <!ELEMENT Title (#PCDATA)>
    <!ELEMENT Year (#PCDATA)>
]>

```

Figure 11.13: A DTD for movies

and its matched closing tag will appear.

The elements inside a sequence or choice can have `minOccurs` and `maxOccurs` attributes to govern how many times they can appear. In the case of a choice, only one of the elements can appear at all, but it can appear more than once if it has a value of `maxOccurs` greater than 1. The rules for `xs:all` are different. It is not permitted to have a `maxOccurs` value other than 1, but `minOccurs` can be either 0 or 1. In the former case, the element might not appear at all.

11.4.4 Attributes

A complex type can have attributes. That is, when we define a complex type T , we can include instances of element `<xs:attribute>`. When we use T as the type of an element E , then E can have (or must have) an instance of this attribute. The form of an attribute definition is:

```

<xs:attribute name = attribute name type = type name
               other information about the attribute />

```

The “other information” may include information such as a default value and usage (required or optional — the latter is the default).

Example 11.14: The notation

```

<xs:attribute name = "year" type = "xs:integer"
              default = "0" />

```

defines `year` to be an attribute of type integer. We do not know of what element `year` is an attribute; it depends where the above definition is placed. The default value of `year` is 0, meaning that if an element without a value for attribute `year` occurs in a document, then the value of `year` is taken to be 0.

As another instance:

```

<xs:attribute name = "year" type = "xs:integer"
              use = "required" />

```


is another definition of the attribute `year`. However, setting `use` to `required` means that any element of the type being defined must have a value for attribute `year`. □

Attribute definitions are placed within a complex-type definition. In the next example, we rework Example 11.13 by making the type `movieType` have attributes for the title and year, rather than subelements for that information.

```

1)  <? xml version = "1.0" encoding = "utf-8" ?>
2)  <xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">

3)      <xs:complexType name = "movieType">
4)          <xs:attribute name = "title" type = "xs:string"
5)              use = "required" />
6)          <xs:attribute name = "year" type = "xs:integer"
7)              use = "required" />
8)      </xs:complexType>

9)      <xs:element name = "Movies">
10)         <xs:complexType>
11)             <xs:sequence>
12)                 <xs:element name = "Movie" type = "movieType"
13)                     minOccurs = "0" maxOccurs = "unbounded" />
14)             </xs:sequence>
15)         </xs:complexType>
16)     </xs:element>

17) </xs:schema>

```

Figure 11.14: Using attributes in place of simple elements

Example 11.15: Figure 11.14 shows the revised XML Schema definition. At lines (4) and (5), the attributes `title` and `year` are defined to be required attributes for elements of type `movieType`. When element `Movie` is given that type at line (10), we know that every `<Movie>` element must have values for `title` and `year`. Figure 11.15 shows the DTD resembling Fig. 11.14. □

11.4.5 Restricted Simple Types

It is possible to create a restricted version of a simple type such as integer or string by limiting the values the type can take. These types can then be used as the type of an attribute or element. We shall consider two kinds of restrictions here:

```

<!DOCTYPE Movies [
  <!ELEMENT Movies (Movie*)>
  <!ELEMENT Movie EMPTY>
  <!--ATTLIST Movie
    title CDATA #REQUIRED
    year  CDATA #REQUIRED
  -->
]>

```

Figure 11.15: DTD equivalent for Fig. 11.14

1. Restricting numerical values by using `minInclusive` to state the lower bound, `maxInclusive` to state the upper bound.⁴
2. Restricting values to an enumerated type.

The form of a range restriction is shown in Fig. 11.16. The restriction has a base, which may be a primitive type (e.g., `xs:string`) or another simple type.

```

<xs:simpleType name = type name >
  <xs:restriction base = base type >
    upper and/or lower bounds
  </xs:restriction>
</xs:simpleType>

```

Figure 11.16: Form of a range restriction

Example 11.16: Suppose we want to restrict the year of a movie to be no earlier than 1915. Instead of using `xs:integer` as the type for element `Year` in line (6) of Fig. 11.12 or for the attribute `year` in line (5) of Fig. 11.14, we could define a new simple type as in Fig. 11.17. The type `movieYearType` would then be used in place of `xs:integer` in the two lines cited above. □

Our second way to restrict a simple type is to provide an enumeration of values. The form of a single enumerated value is:

```

<xs:enumeration value = some value />

```

A restriction can consist of any number of these values.

⁴The “inclusive” means that the range of values includes the given bound. An alternative is to replace `Inclusive` by `Exclusive`, meaning that the stated bounds are just outside the permitted range.

```

<xs:simpleType name = "movieYearType">
  <xs:restriction base = "xs:integer">
    <xs:minInclusive value = "1915" />
  </xs:restriction>
</xs:simpleType>

```

Figure 11.17: A type that restricts integer values to be 1915 or greater

Example 11.17: Let us design a simple type suitable for the genre of movies. In our running example, we have supposed that there are only four possible genres: comedy, drama, sciFi, and teen. Figure 11.18 shows how to define a type `genreType` that could serve as the type for an element or attribute representing our genres of movies. $\square$

```

<xs:simpleType name = "genreType">
  <xs:restriction base = "xs:string">
    <xs:enumeration value = "comedy" />
    <xs:enumeration value = "drama" />
    <xs:enumeration value = "sciFi" />
    <xs:enumeration value = "teen" />
  </xs:restriction>
</xs:simpleType>

```

Figure 11.18: A enumerated type in XML Schema

11.4.6 Keys in XML Schema

An element can have a key declaration, which says that when we look at a certain class C of elements, values of one or more given *fields* within those elements are unique. The concept of “field” is actually quite general, but the most common case is for a field to be either a subelement or an attribute. The class C of elements is defined by a “selector.” Like fields, selectors can be complex, but the most common case is a sequence of one or more element names, each a subelement of the one before it. In terms of a tree of semistructured data, the class is all those nodes reachable from a given node by following a particular sequence of arc labels.

Example 11.18: Suppose we want to say, about the semistructured data in Fig. 11.1, that among all the nodes we can reach from the root by following a *star* label, what we find following a further *name* label leads us to a unique value. Then the “selector” would be *star* and the “field” would be *name*. The implication of asserting this key is that within the root element shown, there

cannot be two stars with the same name. If movies had names instead of titles, then the key assertion would not prevent a movie and a star from having the same name. Moreover, if there were actually many elements like the tree of Fig. 11.1 found in one document (e.g., each of the objects we called “Root” in that figure were actually a single movie and its stars), then different trees could have the same star name without violating the key constraint. $\square$

The form of a key declaration is

```
<xs:key name = key name >
  <xs:selector xpath = path description >
  <xs:field xpath = path description >
</xs:key>
```

There can be more than one line with an `xs:field` element, in case several fields are needed to form the key. An alternative is to use the element `xs:unique` in place of `xs:key`. The difference is that if “key” is used, then the fields must exist for each element defined by the selector. However, if “unique” is used, then they might not exist, and the constraint is only that they are unique if they exist.

The selector path can be any sequence of elements, each of which is a subelement of the previous. The element names are separated by slashes. The field can be any subelement of the last element on the selector path, or it can be an attribute of that element. If it is an attribute, then it is preceded by the “at-sign.” There are other options, and in fact, the selector and field can be any XPath expressions; we take up the XPath query language in Section 12.1.

Example 11.19: In Fig. 11.19 we see an elaboration of Fig. 11.12. We have added the element `Genre` to the definition of `movieType`, in order to have a nonkey subelement for a movie. Lines (3) through (10) define `genreType` as in Example 11.17. The `Genre` subelement of `movieType` is added at line (15).

The definition of the `Movies` element has been changed in lines (24) through (28) by the addition of a key. The name of the key is `movieKey`; this name will be used if it is referenced by a foreign key, as we shall discuss in Section 11.4.7. Otherwise, the name is irrelevant. The selector path is just `Movie`, and there are two fields, `Title` and `Year`. The meaning of this key declaration is that, within any `Movies` element, among all its `Movie` subelements, no two can have both the same title and the same year, nor can any of these values be missing. Note that because of the way `movieType` was defined at lines (13) and (14), with no values for `minOccurs` or `maxOccurs` for `Title` or `Year`, the defaults, 1, apply, and there must be exactly one occurrence of each. $\square$

11.4.7 Foreign Keys in XML Schema

We can also declare that an element has, perhaps deeply nested within it, a field or fields that serve as a reference to the key for some other element. This

```
1)  <? xml version = "1.0" encoding = "utf-8" ?>
2)  <xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">

3)  <xs:simpleType name = "genreType">
4)    <xs:restriction base = "xs:string">
5)      <xs:enumeration value = "comedy" />
6)      <xs:enumeration value = "drama" />
7)      <xs:enumeration value = "sciFi" />
8)      <xs:enumeration value = "teen" />
9)    </xs:restriction>
10) </xs:simpleType>

11) <xs:complexType name = "movieType">
12)   <xs:sequence>
13)     <xs:element name = "Title" type = "xs:string" />
14)     <xs:element name = "Year" type = "xs:integer" />
15)     <xs:element name = "Genre" type = "genreType"
16)       minOccurs = "0" maxOccurs = "1" />
17)   </xs:sequence>
18) </xs:complexType>

19) <xs:element name = "Movies">
20)   <xs:complexType>
21)     <xs:sequence>
22)       <xs:element name = "Movie" type = "movieType"
23)         minOccurs = "0" maxOccurs = "unbounded" />
24)     </xs:sequence>
25)   </xs:complexType>
26)   <xs:key name = "movieKey">
27)     <xs:selector xpath = "Movie" />
28)     <xs:field xpath = "Title" />
29)     <xs:field xpath = "Year" />
30)   </xs:key>
31) </xs:element>

32) </xs:schema>
```

Figure 11.19: A schema for movies in XML Schema

capability is similar to what we get with ID's and IDREF's in a DTD (see Section 11.3.4). However, the latter are untyped references, while references in XML Schema are to particular types of elements. The form of a foreign-key definition in XML Schema is:

```
<xs:keyref name = foreign-key name refer = key name >
  <xs:selector xpath = path description >
  <xs:field xpath = path description >
</xs:keyref>
```

The schema element is `xs:keyref`. The foreign-key itself has a name, and it refers to the name of some key or unique value. The selector and field(s) are as for keys.

Example 11.20: Figure 11.20 shows the definition of an element `<Stars>`. We have used the style of XML Schema where each complex type is defined within the element that uses it. Thus, we see at lines (4) through (6) that a `<Stars>` element consists of one or more `<Star>` subelements.

At lines (7) through (11), we see that each `<Star>` element has three kinds of subelements. There is exactly one `<Name>` and one `<Address>` subelement, and any number of `<StarredIn>` subelements. In lines (12) through (15), we find that a `<StarredIn>` element has no subelements, but it does have two attributes, `title` and `year`.

Lines (22) through (26) define a foreign key. In line (22) we see that the name of this foreign-key constraint is `movieRef` and that it refers to the key `movieKey` that was defined in Fig. 11.19. Notice that this foreign key is defined within the `<Stars>` definition. The selector is `Star/StarredIn`. That is, it says we should look at every `<StarredIn>` subelement of every `<Star>` subelement of a `<Stars>` element. From that `<StarredIn>` element, we extract the two fields `title` and `year`. The `@` indicates that these are attributes rather than subelements. The assertion made by this foreign-key constraint is that any title-year pair we find in this way will appear in some `<Movie>` element as the pair of values for its subelements `<Title>` and `<Year>`. $\square$

11.4.8 Exercises for Section 11.4

Exercise 11.4.1: Give an example of a document that conforms to the XML Schema definition of Fig. 11.12 and an example of one that has all the elements mentioned, but does not conform to the definition.

Exercise 11.4.2: Rewrite Fig. 11.12 so that there is a named complex type for `Movies`, but no named type for `Movie`.

Exercise 11.4.3: Write the XML Schema definitions of Fig. 11.19 and 11.20 as a DTD.

```

1) <? xml version = "1.0" encoding = "utf-8" ?>
2) <xs:schema xmlns:xs = "http://www.w3.org/2001/XMLSchema">

3) <xs:element name = "Stars">

4)     <xs:complexType>
5)         <xs:sequence>
6)             <xs:element name = "Star" minOccurs = "1"
              maxOccurs = "unbounded">
7)                 <xs:complexType>
8)                     <xs:sequence>
9)                         <xs:element name = "Name"
                            type = "xs:string" />
10)                        <xs:element name = "Address"
                            type = "xs:string" />
11)                        <xs:element name = "StarredIn"
                            minOccurs = "0"
                            maxOccurs = "unbounded">
12)                            <xs:complexType>
13)                                <xs:attribute name = "title"
                                    type = "xs:string" />
14)                                <xs:attribute name = "year"
                                    type = "xs:integer" />
15)                            </xs:complexType>
16)                        </xs:element>
17)                    </xs:sequence>
18)                </xs:complexType>
19)            </xs:element>
20)        </xs:sequence>
21)    </xs:complexType>

22)    <xs:keyref name = "movieRef" refers = "movieKey">
23)        <xs:selector xpath = "Star/StarredIn" />
24)        <xs:field xpath = "@title" />
25)        <xs:field xpath = "@year" />
26)    </xs:keyref>

27) </xs:element>

```

Figure 11.20: Stars with a foreign key

11.5 Summary of Chapter 11

- ◆ *Semistructured Data*: In this model, data is represented by a graph. Nodes are like objects or values of attributes, and labeled arcs connect an object to both the values of its attributes and to other objects to which it is connected by a relationship.
- ◆ *XML*: The Extensible Markup Language is a World-Wide-Web Consortium standard that represents semistructured data linearly.
- ◆ *XML Elements*: Elements consist of an opening tag `<Foo>`, a matched closing tag `</Foo>`, and everything between them. What appears can be text, or it can be subelements, nested to any depth.
- ◆ *XML Attributes*: Tags can have attribute-value pairs within them. These attributes provide additional information about the element with which they are associated.
- ◆ *Document Type Definitions*: The DTD is a simple, grammatical form of defining elements and attributes of XML, thus providing a rudimentary schema for those XML documents that use the DTD. An element is defined to have a sequence of subelements, and these elements can be required to appear exactly once, at most once, at least once, or any number of times. An element can also be defined to have a list of required and/or optional attributes.
- ◆ *Identifiers and References in DTD's*: To represent graphs that are not trees, a DTD allows us to declare attributes of type ID and IDREF(S). An element can thus be given an identifier, and that identifier can be referred to by other elements from which we would like to establish a link.
- ◆ *XML Schema*: This notation is another way to define a schema for certain XML documents. XML Schema definitions are themselves written in XML, using a set of tags in a namespace that is provided by the World-Wide-Web Consortium.
- ◆ *Simple Types in XML Schema*: The usual sorts of primitive types, such as integers and strings, are provided. Additional simple types can be defined by restricting a simple type, such as by providing a range for values or by giving an enumeration of permitted values.
- ◆ *Complex Types in XML Schema*: Structured types for elements may be defined to be sequences of elements, each with a minimum and maximum number of occurrences. Attributes of an element may also be defined in its complex type.
- ◆ *Keys and Foreign Keys in XML Schema*: A set of elements and/or attributes may be defined to have a unique value within the scope of some

enclosing element. Other sets of elements and/or attributes may be defined to have a value that appears as a key within some other kind of element.

11.6 References for Chapter 11

Semistructured data as a data model was first studied in [5] and [4]. LOREL, the prototypical query language for this model is described in [3]. Surveys of work on semistructured data include [1], [7], and the book [2].

XML is a standard developed by the World-Wide-Web Consortium. The home page for information about XML is [9]. References on DTD's and XML Schema are also found there. For XML parsers, the definition of DOM is in [8] and for SAX it is [6]. A useful place to go for quick tutorials on many of these subjects is [10].

1. S. Abiteboul, "Querying semi-structured data," *Proc. Intl. Conf. on Database Theory* (1997), Lecture Notes in Computer Science 1187 (F. Afrati and P. Kolaitis, eds.), Springer-Verlag, Berlin, pp. 1–18.
2. S. Abiteboul, D. Suciu, and P. Buneman, *Data on the Web: From Relations to Semistructured Data and XML*, Morgan-Kaufmann, San Francisco, 1999.
3. S. Abiteboul, D. Quass, J. McHugh, J. Widom, and J. L. Weiner, "The LOREL query language for semistructured data," In *J. Digital Libraries* 1:1, 1997.
4. P. Buneman, S. B. Davidson, and D. Suciu, "Programming constructs for unstructured data," *Proceedings of the Fifth International Workshop on Database Programming Languages*, Gubbio, Italy, Sept., 1995.
5. Y. Papakonstantinou, H. Garcia-Molina, and J. Widom, "Object exchange across heterogeneous information sources," *IEEE Intl. Conf. on Data Engineering*, pp. 251–260, March 1995.
6. Sax Project, <http://www.saxproject.org/>
7. D. Suciu (ed.) Special issue on management of semistructured data, *SIGMOD Record* 26:4 (1997).
8. World-Wide-Web Consortium, <http://www.w3.org/DOM/>
9. World-Wide-Web Consortium, <http://www.w3.org/XML/>
10. W3 Schools, <http://www.w3schools.com>

